

Career Path Recommendation Engine (Role-to-Role + Skill Gap + Graph Path)

1. Objective

Build a recommendation module that:

- suggests similar roles for a given job role,
- generates a step-wise career progression,
- provides skill-gap analysis for each transition,

2. Dataset Used

JobRoleToSkillRecommendation.xlsx

Key columns used:

- Industry, Department, Job Role
- Skill Title, Skill Category, Proficiency Level
(plus other descriptive columns for context)

3. Tech Stack / Libraries

- **Python**
- **pandas / numpy**: data loading, cleaning, transformation
- **scikit-learn**: cosine similarity for role similarity scoring
- **FastAPI**: API creation
- **uvicorn**: local server run
- **networkx + matplotlib**: graph visualization (optional UI)
- **openpyxl** (via pandas): reading Excel

4. Data Preprocessing (What is done)

- Load Excel dataset
- Normalize column names
- Build a **Role–Skill matrix**
 - Rows = Job Roles
 - Columns = Skills
 - Values = Proficiency Level (numeric mapping)
- Handle missing values / duplicates
- Create role list for resolver

5. Core Logic (How recommendation works)

5.1 Role Resolution

- User input role may be lowercase/spelling variant
- Match against dataset role names using close matching
- Output:
 - resolved_role
 - suggestions list (if close matches exist)

5.2 Role Similarity (Cosine Similarity)

- Each role converted to a skill vector (from role-skill matrix)
- Similarity between roles computed using **cosine similarity**
- Higher score = more similar skill requirement

5.3 Skill Gap Calculation

For transition: **from_role** → **to_role**

- Compare proficiency levels skill-wise
- $\text{gap} = (\text{target skill level} - \text{current skill level})$
- Return top missing skills with highest gaps

6. API Endpoints Implemented

/recommend

- Purpose: Similar role suggestions
- Output: suggestions list based on similarity

/career_path

- Purpose: Step-wise progression (Role A → Role B → Role C)
- Output: path steps + match_score + missing_skills

/graph_path

- Purpose: Graph-based path representation
- Output: transitions as edges with edge_weight (similarity)

7. Execution (How to run locally)

1. Create venv and install requirements
2. Run server:

```
uvicorn src.api:app --reload
```

3. Open:

<http://127.0.0.1:8000/docs>

Optional graph:

```
python -m src.test_plot
```

8. Results / Example

Input: Data Analyst

Output:

- recommended next roles
- career path steps
- missing skills per step
- graph transitions + visualization

9. Limitations (Honest but safe)

- Recommendations depend on dataset quality and role coverage
- If role not present in dataset, system prompts for correct role spelling or suggestions
- Graph edges are based on skill similarity (not real-world promotion probability)

10. Future Enhancements

- Add UI page (HTML) to avoid long URLs
- Add “experience level” filter
- Add real transition history if available (true graph transitions)